

AUDITORIA — BLOQUEIO/CLASSIFICAÇÃO INTERNA DE IMAGENS DE REFERÊNCIA

Varredura forense, correções e delegação da validação de conteúdo ao provedor

Data: 2026-07-26 (~04:40-04:46Z, UTC)

Projeto: Fluxyra (Next.js 16 / TypeScript / App Router)

Branch: feat/f3-piapi-landing-pricing | **Deploy:** <https://5be3213ef.abacusai.cloud>
(fluxyra.service , porta 3000)

Commit desta correção: 60dce74

Provider de vídeo: PiAPI (14 modelos)

RESUMO EXECUTIVO: Foram encontradas **2 ocorrências de lógica INDEVIDA** que classificavam/substituíam por conta própria a resposta do provedor: (1) substituição das mensagens de erro da PiAPI por textos hardcoded em `status/route.ts`; (2) injeção de `auto_upload_assets: true` como workaround de content restriction em `client.ts`. **Ambas foram removidas.** Nenhuma biblioteca de detecção de rosto/pessoa, moderação ou classificação de imagem foi encontrada. As validações **técnicas** (MIME, tamanho) foram confirmadas como legítimas e **mantidas intactas**. O sistema agora **delega 100% da validação de conteúdo ao provedor** e **preserva a mensagem original** da PiAPI.

⚠ Nenhum vídeo foi gerado e **nenhuma task foi criada na PiAPI** durante esta auditoria. Nenhum secret é exposto neste relatório.

PARTE A — VARREDURA FORENSE

Varredura em todo `src/` (frontend, backend, API routes, libs, hooks, stores, middleware) por: bloqueio por conteúdo, classificação de imagem, filtros por “real person / celebrity / deepfake / content restriction / face / person detection”, substituição da resposta do provider por texto hardcoded, e omissão/modificação de `logs[] / error.message`.

A.1 Ocorrências INDEVIDAS encontradas (lógica de conteúdo/classificação)

● INDEVIDA #1 — Substituição das mensagens do provider por texto hardcoded

- **Arquivo:** `src/app/api/generate/status/route.ts`
- **Função:** handler `GET` (bloco `if (state === "failed")`)
- **Linhas (antes da correção):** 231-251
- **Trecho:**

```
const logsText = logs.join(" \n ").toLowerCase();
let errMsg = providerMsg;
if (logsText.includes("real person") || logsText.includes("content restriction")) {
  errMsg = "Imagem rejeitada: a foto contém rosto de pessoa real. O Seedance não aceita imagens com pessoas reais por restrições de deepfake. Use uma imagem sem rostos visíveis.";
} else if (logsText.includes("plan limit") || logsText.includes("active task count")) {
  errMsg = "Limite de tarefas simultâneas atingido. Aguarde a conclusão das gerações em andamento e tente novamente.";
} else if (logsText.includes("nsfw") || logsText.includes("content policy")) {
  errMsg = "Conteúdo rejeitado pela política do provedor. Revise o prompt ou a imagem de referência.";
}
```

- **Motivo original da implementação:** no commit `a93267b` (auditoria anterior), tentou-se “traduzir” o erro genérico da PiAPI (`Internal error. Please try again later.`) em mensagens PT-BR mais claras, lendo palavras-chave dentro de `logs[]` .
- **Impacto no fluxo:** o sistema **interpretava e reescrevia** a causa reportada pelo provedor, inclusive **afirmando por conta própria** que a imagem “contém rosto de pessoa real” e que “o Seedance não aceita pessoas reais”. Isso: (a) mascara a mensagem real do provedor; (b) insere uma **classificação de conteúdo interna** (mesmo que derivada dos logs, é o Fluxyra decidindo a semântica e trocando o texto); (c) pode enganar o usuário quando o log não corresponder exatamente à heurística. **É exatamente o comportamento que deve ser removido.**

● INDEVIDA #2 — `auto_upload_assets: true` como workaround de content restriction

- **Arquivo:** `src/lib/piapi/client.ts`
- **Função:** `buildVideoPayloadInner` (branch do backend Seedance)
- **Linhas (antes da correção):** 418-422
- **Trecho:**

```
// Quando há imagens de referência, a PiAPI pode rejeitar fotos com rosto de
// pessoa real ("content restriction"). Definir auto_upload_assets:true faz a
// PiAPI hospedar/reprocessar a imagem internamente, contornando a restrição
// (conforme instrução retornada nos logs da própria PiAPI).
if (imgUrls.length > 0) seedancePayload.auto_upload_assets = true;
```

- **Motivo original da implementação:** os próprios `logs[]` da PiAPI sugeriam `auto_upload_assets: true` para task types “-less-restriction”; foi adicionado como tentativa de **contornar** o bloqueio anti-deepfake.
- **Averiguação (era workaround ou parâmetro legítimo?): é workaround.** Não é parâmetro oficial/documentado do Seedance para o task type usado (`seedance-2`); a dica da PiAPI refere-se a task types “-less-restriction”. A auditoria da Etapa 1 (relatório `AUDITORIA_ETAPA1_COMPLEMENTO.md` , §4.3) **comprovou por evidência crua** que tasks enviadas COM `auto_upload_assets: true` (ex.: `59d36e05` , `fc6b1799` , `44cf6344` , `3f4de519`) **continuaram falhando com a mesma restrição**. Ou seja, além de indevido, **não funcionava**.
- **Impacto no fluxo:** o Fluxyra tentava manipular a decisão de aceite do provedor em vez de deixá-lo julgar; sem efeito prático e mascarando a natureza da rejeição.

A.2 Locais verificados e SEM lógica indevida (delegam corretamente ao provider)

Arquivo	Verificação	Resultado
<code>src/app/api/webhooks/piapi/route.ts</code>	tratamento de <code>status=failed</code>	✅ usa <code>error \\ \\ "Erro no provedor de IA"</code> — preserva a mensagem original
<code>src/app/api/generate/image/route.ts</code>	catch de erro do provider	✅ persiste <code>String(apiError)</code> — sem classificação
<code>src/app/api/generate/video/route.ts</code>	submissão	✅ nenhuma validação/filtro de conteúdo; estorno em caso de erro
<code>src/components/studio/generation-dock.tsx</code>	upload de referência (<code>uploadReferenceFile</code>)	✅ apenas envia ao <code>/api/upload</code> ; sem classificação
<code>src/components/studio/studio-feed.tsx</code> (<code>FailedCard</code>)	exibição de falha	⚠️ apenas remove prefixos técnicos (<code>PiAPIError: / AbacusImageError: </code>) e trunca para exibição compacta — não altera semântica; a mensagem completa fica disponível
<code>src/middleware.ts</code> / <code>src/lib/supabase/middleware.ts</code>	auth/sessão	✅ sem filtro de conteúdo
Busca por libs (<code>face-api</code> , <code>nsfwjs</code> , <code>rekognition</code> , <code>moderation</code> , <code>@tensorflow</code> , <code>sightengine</code> , <code>clarifai</code> , <code>detectFace</code> , etc.)	repositório inteiro	✅ nenhuma ocorrência

A.3 Validações LEGÍTIMAS (técnicas) — CONFIRMADAS e MANTIDAS

- **Arquivo:** `src/app/api/upload/route.ts`
- `MAX_BYTES = 20 * 1024 * 1024` (limite de 20 MB) — l.10, aplicado em l.49.
- `ALLOWED_PREFIXES = ["image/", "video/", "audio/"]` (tipos MIME permitidos) — l.12, aplicado em l.57.
- Mapa de extensão por MIME (`EXT_BY_MIME`) — l.14+.
- **Classificação:** LEGÍTIMA (técnica). Não filtra por conteúdo, apenas formato/tamanho. **Mantidas intactas.**
- **Nginx:** `client_max_body_size 25m` (`deploy/5be3213ef.conf`) — limite de corpo da requisição, técnico. Mantido.

- `generation-dock.tsx` : atributos `accept="image/*|video/*|audio/*"` nos inputs — filtro de tipo no seletor de arquivo, técnico. Mantido.

PARTE B — CORREÇÕES APLICADAS

Correção #1 — Preservar a mensagem original do provider (`status/route.ts`)

Removida toda a lógica de substituição por texto hardcoded. Agora concatena `error.message` + `logs[]` e apresenta com prefixo neutro, **sem alterar o conteúdo**.

Antes (I.231-251): ver trecho em A.1 (#1).

Depois (I.231-241):

```
// NÃO classificamos nem substituímos a mensagem do provedor por texto
// interno. Preservamos SEMPRE a mensagem original (error.message +
// logs[] concatenados quando houver), com um prefixo neutro em PT-BR.
// A validação de conteúdo é responsabilidade exclusiva do provedor.
const providerDetail = [providerMsg, ...logs]
  .map((s) => String(s).trim())
  .filter(Boolean)
  .join(" | ");
const errMsg = providerDetail
  ? `0 provider rejeitou a solicitação: ${providerDetail}`
  : "0 provider rejeitou a solicitação.";
```

O `errMsg` resultante é persistido em `generations.error_message` e retornado ao frontend (`return NextResponse.json({ status: "failed", error_message: String(errMsg) })`), preservando integralmente `error.message` e todos os itens de `logs[]`.

Correção #2 — Remover `auto_upload_assets: true` (`client.ts`)

Removida a injeção do flag. Comentário documenta o motivo (workaround não-oficial e comprovadamente ineficaz).

Antes (I.418-422): ver trecho em A.1 (#2).

Depois (I.418-424):

```
// OBS.: NÃO enviamos `auto_upload_assets: true`. Esse flag havia sido
// adicionado como tentativa de contornar a restrição de conteúdo ("real
// person") da PiAPI, mas (a) não é um parâmetro oficial/documentado do
// Seedance e (b) comprovadamente NÃO contorna o bloqueio (auditoria da
// Etapa 1: tasks continuaram falhando com o mesmo erro mesmo com o flag).
// A decisão de aceitar ou rejeitar a imagem é 100% do provedor.
return seedancePayload;
```

Correção #3 — Outras lógicas de filtragem de conteúdo

Nenhuma outra encontrada na varredura (A.2/A.3). Nada mais a remover.

Correção #4 — Validações técnicas legítimas

`MAX_BYTES`, `ALLOWED_PREFIXES`, `EXT_BY_MIME`, `client_max_body_size`, `accept="..."` — **todas preservadas**, sem alteração.

Correção #5 — Log e retorno da resposta original do provider

- **Log:** `status/route.ts` continua registrando via `audit-log.ts` o evento `task_failed_piapi` com `provider_error` (= `error.message`) e `piapi_logs` (= `logs[]` completos, sem filtro), correlacionado por `requestId` em `logs/app.log`. A instrumentação forense de `client.ts` (`piapiFetch` / `getTaskStatus`) também loga a resposta crua da PiAPI (HTTP status, body, `logs[]`).
- **Retorno ao frontend:** `error_message` agora carrega `error.message` + `logs[]` originais (prefixo neutro), sem substituição.

VALIDAÇÃO (PARTE C)

Passo	Resultado
<code>npx tsc --noEmit</code>	rc=0 (sem erros de tipo)
<code>npm run build</code>	rc=0 (build OK)
<code>sudo systemctl restart fluxyra.service @ 04:45:58Z</code>	<code>active</code>
<code>curl localhost:3000/</code>	HTTP 200 (0,19s)
<code>git commit</code>	60dce74 (2 arquivos alterados)

FLUXO ATUAL CORRIGIDO

Usuário

- Upload de referência (`/api/upload`)
 - validação TÉCNICA apenas: MIME (image/ | video/ | audio/) + tamanho (≤ 20 MB)
 - SEM classificação de conteúdo, SEM detecção de rosto/pessoa
- Backend `/api/generate/video`
 - valida auth, prompt, modelo, créditos (nada de conteúdo)
 - `buildVideoPayload` monta payload SEM `auto_upload_assets`
- Provider (PiAPI) ← ÚNICO responsável por aceitar/rejeitar o conteúdo
- `/api/generate/status` lê a resposta
 - preserva `error.message` + `logs[]` originais (prefixo neutro "0 provider rejeitou a solicitação:")
 - loga resposta crua em `logs/app.log` (`requestId`)
- Frontend exibe a mensagem ORIGINAL do provedor (sem texto interno substituto)

Confirmação: o sistema **não bloqueia, não classifica e não reescreve** o conteúdo/semântica das respostas do provedor. A validação de conteúdo é **integralmente delegada à PiAPI**, e a mensagem original é **preservada** do backend até a UI.

PENDENTE DE TESTE MANUAL

A verificação end-to-end exige gerar de fato (o que cria task na PiAPI) e, portanto, deve ser feita manualmente pelo usuário. Cenários sugeridos:

1. **Imagem de pessoa real no Seedance 2.0:** em <https://5be3213ef.abacusai.cloud/studio> (aba Vídeo → Seedance 2.0), anexe uma foto com rosto de pessoa real e gere. **Verificar:** a mensagem exibida na UI deve começar com “O provider rejeitou a solicitação:” seguida do texto **exato** vindo da PiAPI (ex.: `Internal error. Please try again later. | mode auto-detected... | The request was rejected because the input image may contain a real person...`), **sem** o texto interno antigo (“Imagem rejeitada: a foto contém rosto de pessoa real...”). Conferir também em `logs/app.log` o evento `task_failed_piapi` com `provider_error` e `piapi_logs` completos.
2. **Imagem sem pessoas (produto/paisagem) no Seedance:** deve gerar normalmente — confirma que a remoção do `auto_upload_assets` não quebrou o caminho feliz.
3. **Limite de concorrência (plan limit):** disparar 3 gerações Seedance em sequência rápida. **Verificar:** a mensagem de falha deve conter o texto original do provedor (ex.: `active task count 2 has reached the plan limit`), sem substituição interna.
4. **Erro de política/NSFW (se aplicável):** gerar com conteúdo que o provedor recuse por política. **Verificar:** a UI mostra a mensagem original do provedor, não o texto interno antigo.
5. **Correlação de logs:** para qualquer falha acima, capturar o `requestId` no console do navegador (`[FLUXYRA-GEN]`) e localizar as linhas `[FLUXYRA-AUDIT]` correspondentes em `logs/app.log`, confirmando que `error.message` e `logs[]` chegam íntegros do provider ao backend e à UI.

Observação: o `FailedCard` (feed) trunca a mensagem para exibição compacta e remove apenas o prefixo técnico `PiAPIError:`. A mensagem completa (original) fica em `generations.error_message` e pode ser exibida integralmente no lightbox/detalhe. Se desejado, um passo futuro pode exibir a mensagem completa sem truncamento na UI de falha.